

程序设计实习 课程项目报告

—— Plague Inc.（瘟疫公司）复刻

项目名称 Plague Inc.（瘟疫公司）桌面版

开发环境 Qt 6 + C++17, CMake 构建系统

小组成员 宋佳程、蒋昊辰、王梓存

提交日期 2026 年 7 月

一、程序功能介绍

1.1 游戏概述

本作品是基于经典策略模拟游戏《Plague Inc.（瘟疫公司）》的桌面端复刻，使用 C++ 与 Qt 6 框架开发。玩家扮演一种病原体，通过在全球范围内传播感染，目标是在人类研发出解药之前消灭全人类。游戏复刻了原作的核心玩法、数值体系、UI 交互与新闻事件系统，并在海空交通可视化上做出了具有特色的实现。

1.2 核心玩法

打开游戏后，玩家先选择难度，为病原体命名，然后在世界地图上选择一个地区投放“零号病人”。游戏时间逐日推进。DNA 点数是升级病原体的唯一资源——可通过新感染一个地区获得一次性奖励、点击地图上定期刷新的 DNA 气泡、以及 DNA 随时间自行缓慢增长等方式获取。

DNA 点数用于在疾病菜单中进化病原体的技能树。技能树分为三大分支：传播途径（影响病毒扩散方式，如空气/水源传播、动物媒介）、发病症状（提升传染性、严重性和致命性）、特殊能力（提供抗药性、基因重组等复合效果）。每个技能以六边形节点呈现在蜂窝布局中，需依次解锁前置技能。

游戏过程中新闻系统会弹出疫情报告提示进展，玩家需观察地图上感染的扩散动态，在解药研发完成前实现全球感染。

1.3 特色机制

（1）海空交通传播可视化：

机场与港口的航线上实时播放飞机与轮船运行动画。出发国出现感染时，出发载具有概率变为红色并在目标地区投放外来病例，实现跨海传播。被感染载具经过的航线留下渐深的红色轨迹。

（2）政府封锁机制：

各国政府根据恐慌指数（由感染比例、严重性和致命性计算）依次关闭机场、港口和陆地边境。封锁后停发载具、拒收外来病例，设施图标叠加红叉标记。各地区的封锁阈值由高斯分布随机生成，使每局游戏的封锁行为具有差异性。

（3）解药研发系统：

全球科学家根据恐慌程度和存活人口比例自动研发解药，其速度受病原体抗药复杂性影响。“基因重组”技能可使解药进度一次性倒退 5%~10%。若解药进度达 100% 而人类尚未灭绝，则人类获胜。

二、项目各模块与类设计细节

2.1 项目架构总览

本项目遵循“模型-视图分离”的设计思想。GameCore 作为唯一的 game 数据权威，通过 Qt 信号-槽机制向所有 UI 组件广播状态变更。各界面组件只负责显示与交互，不直接修改 game 数据，保证了逻辑与界面层清晰解耦。核心组件包括：GameCore（游戏逻辑引擎）、MapWidget（地图与交通仿真）、DiseaseMenu（疾病菜单与技能树）、WorldMenu（世界概况统计）、以及 PopulationBar、HudPanel、NoticePopup 等辅助组件。

2.2 GameCore —— 游戏核心逻辑

GameCore 继承自 QObject，管理 58 个地区的感染/死亡/人口数据（RegionData 结构体）、疾病全局属性（传染性/严重性/致命性）、DNA 点数、解药进度和技能树解锁状态。其核心是每日结算循环 onTick()，依次进行以下计算：

- 地区内部感染增长：基于传染性和当前感染人数计算新增病例
- 陆地接壤传播：按“基础边境流量 × 源国感染比例 × 传染性 × 途径乘区 × 边境状态”计算每日突破概率
- 每日死亡结算：按“单日死亡率 = 基础致死倍率 × 致命性 + 保底死亡人数”计算
- 解药进度推进：基于全球恐慌度和存活比例计算每日研发增量
- 政府封锁判定：更新各地区的机场/港口/边境封锁状态
- 疾病被发现判定和游戏结束判定

GameCore 提供 seedInfection（投毒）、evolveSkill（进化技能）、devolveSkill（退化技能）、collectDNA（收集 DNA）、setSpeed（速度控制）等公共接口，并通过 dayPassed、globalStatsUpdated、regionUpdated、skillsChanged 等信号向界面广播数据变更。

2.3 MapWidget —— 世界地图与交通仿真

MapWidget 继承自 QGraphicsView，是本项目最复杂的 UI 组件，负责地图渲染、感染可视化、DNA 气泡系统和海空交通仿真。

地图渲染方面：

以高清卫星图和等矩形投影 SVG 矢量图为底图，各地区以 QGraphicsPathItem 呈现。感染可视化使用点阵系统——各地区的红/黑色密度点反映感染和死亡比例。支持鼠标滚轮缩放和拖拽平移。

DNA 气泡方面：

新感染地区时生成红色 DNA 气泡（一次性），感染激增时生成橙色气泡。气泡具有生命周期，地图还会定时随机刷新普通气泡。点击即可收集。

海空交通方面：

TransportData 命名空间下定义了全球约 50 个机场/港口节点的航线网络。飞机走直线，轮船走 A* 海洋寻路网格（附带离岸平滑和样条插值）。每日按平均间隔投放载具，出发国感染比例越高，载具“带毒”概率越大。被感染载具经过的航线逐次加深（10 档透明度），飞机实线、轮船虚线。游戏速度变化时载具动画同步调整，暂停时冻结。轮船 A* 寻路是本项目的一个技术亮点：栅格化海陆掩膜、计算离岸距离和连通水域编号，引入离岸代价系数避免贴岸抖动，最终通过样条插值平滑折线。

2.4 DiseaseMenu —— 疾病菜单与技能树

DiseaseMenu 是一个半透明红色渐变覆盖层菜单，分为四个分页：疾病概况、传播途径、发病症状、特殊能力。其子组件包括：

- HexGrid：绘制 10×6/5 交替共 55 格的平顶六边形蜂窝骨架，管理技能节点定位与吸附

- HexNode: 单个六边形节点（继承 QWidget），通过自定义 paintEvent 绘制六边形轮廓、技能图标和状态颜色。支持已解锁（金色描边）、可购买（浅红描边）、未解锁（压暗）和选中（黄色外环）四种状态
 - StatBars: 底部属性状态条，显示 DNA 及传染性/严重性/致命性条形图，支持预览选中技能的属性增量
 - VideoBackground: 使用 QMediaPlayer + QVideoSink 循环播放背景视频（病原体动画）
- skilltree.cpp 是技能数据定义中心，以表格集中管理所有技能的 ID、名称、消耗、属性加成和位置。技能间相邻关系和前置关系通过算法根据蜂窝格子编号自动计算，修改数据时无需手动更新连线拓扑。

2.5 WorldMenu —— 世界概况与数据统计

WorldMenu 是一个半透明蓝色渐变覆盖层菜单，包含两个分页：

- 世界概况页：以扇形统计图（PieChart，继承 QWidget）展示全球人口健康/感染/死亡比例，按状态分列地区名单，以及解药进度展示
- 其他数据页：以折线图（LineChart，继承 QWidget）展示多组历史趋势，包括人口变化曲线、解药进度曲线和疾病属性进化曲线

2.6 辅助模块

PopulationBar（底部人口占比条）：

以黑/红/蓝三色条表示选定地区或全球的死亡/感染/健康人口占比，支持切换显示模式。

HudPanel（HUD 面板）：

将装饰图按宽高比居中绘制并叠加可定制文本槽，用于主界面四个角的信息展示（新闻区、DNA、解药、世界概况），点击发射 clicked 信号。

NoticePopup 与 NoticeManager（新闻系统）：

NoticePopup 实现模态新闻弹窗，弹窗期间自动暂停游戏。NoticeManager 监听 GameCore 信号，按条件从 notice.txt 加载新闻文本触发弹窗，支持占位符替换和仅触发一次的机制。

SoundManager（音频管理）：

全局单例，循环播放背景音乐并提供按钮点击/关闭、气泡产生/收集四种音效。通过 Qt 事件过滤器自动为所有按钮接入音效，无需逐个接线。

数据层设计：

regions.h 管理 58 个地区的名称、人口、接壤关系；GameTypes.h 定义 RegionData、DiseaseStats、HistoryPoint 等数据结构；GameParams.h 以 inline constexpr 常量集中管理所有可调数值，与 formula.md 的五套公式一一对应，实现了数据与逻辑的解耦，平衡调整只需修改该文件。

三、小组成员分工情况

本项目分为初期、中期和后期三个阶段，由宋佳程、蒋昊辰、王梓存三名同学协作完成。

■ 初期——项目搭建

- 王梓存：负责 Qt 模型建构，搭建项目基础框架，包括 CMake 构建配置、主窗口布局 and 核心类接口设计
- 蒋昊辰：负责世界地图绘制与组织，包括 SVG 地图裁剪配准、各地区边界矢量描摹
- 宋佳程：负责项目统筹规划和核心传播代码的初步编写

■ 中期——功能开发

- 王梓存：完成疾病菜单中技能加点页面的完整逻辑和六边形技能树图像构造（HexGrid 蜂窝布局、HexNode 渲染、进化/退化交互流程）
- 蒋昊辰：完成声音系统（背景音乐、音效、按钮声自动挂载）和新闻系统（Popup + Manager 架构与触发逻辑）的设计实现
- 宋佳程：完成海空交通传播系统的核心逻辑（交通数据定义、载具调度与动画、A* 海洋寻路）

■ 后期——整合与打磨

- 三人共同完成功能联调、Bug 修复和体验优化
- 宋佳程统筹素材整合、UI 资源管理、参数平衡性调整和最终代码审查
- 蒋昊辰补充新闻文本和音效微调；王梓存完善技能树视觉效果与交互细节

四、项目总结与反思

4.1 项目成果

本项目较为成功地复刻了《Plague Inc.》的核心体验。在玩法层面，实现了病原体命名、初始地区选择、基于五套数学公式的感染传播模型、技能进化树、政府封锁机制、DNA 经济系统和解药研发系统；在工程层面，实现了 Qt 信号-槽机制与 MVC 架构的合理运用，代码结构清晰、数据与逻辑解耦；在视觉层面，复刻了六边形技能树、感染热力图、新闻弹窗等经典元素，并实现了海空交通可视化这一具有特色的扩展。

4.2 存在的不足

（1）版本控制工具使用不够熟练：

项目初期对 Git 和 GitHub 的协作流程不够熟悉，在分支管理、代码合并、冲突解决方面遇到困难，导致了代码同步滞后和重复工作，影响了开发效率。

（2）进度预估过于乐观：

对技术难点（如海空寻路算法调试、地图配准精度控制）的预估不足，导致后期时间紧张，部分功能（移动端适配、存档系统等）未能实现。这提醒我们在项目规划时应为风险预留充足的缓冲时间。

（3）创新性有待提高：

大部分时间精力集中于对原作的模仿复刻，玩法创新探索不足。除海空交通可视化（原作仅以概率数值体现，本项目以动态载具动画呈现）外，其余核心机制基本遵循原版设计。

4.3 经验与收获

通过本次项目实践，小组成员获得了显著的成长：

- 团队协作方面：体会了 Git 分支管理、任务拆解、接口协商等技能的重要性
- 技术方面：深入理解 C++ 面向对象设计与 Qt 框架的信号-槽机制、事件循环、自定义绘制、多媒体播放等
- 工程实践方面：从零构建传染病数学模型，将抽象公式转化为可执行的代码
- 问题解决方面：在 A* 寻路、坐标转换、动画同步等问题的调试中提升了分析和解决能力

总的来说，本次课程项目不仅完成了对经典游戏的复刻，更让三名成员在真实团队协作中积累了宝贵的软件开发经验，为日后参与更复杂的项目奠定了坚实基础。